Seismic Viewer

Analisi, Progettazione e Implementazione di un sistema interattivo per la visualizzazione 4D di dati sismici

Documento di progetto

Alessio Curcio
Giada Alfonsi
Tutor: Daniele Bailo

Indice

1 Introduzione ed Analisi del Dominio **2**

2 Analisi dei Requisiti **2**
2.1 Requisiti Funzionali Primari 2
2.2 Funzionalità Avanzate 2
2.3 Estensioni ed Ottimizzazioni 2

3 Architettura di Sistema (Tiered Model) **3**
3.1 Data Layer (Python Core) 3
3.2 Logic Layer (Bridge API) 3
3.3 Presentation Layer (WebView UI) 3

4 Infrastruttura di Caricamento (Loader & Pre-computation) **3**
4.1 Normalizzazione Temporale 3
4.2 Visual Data Packaging e Ottimizzazione delle Performance 3

5 Motore di Visualizzazione e Geospazialità **4**
5.1 Formalizzazione della Proiezione di Mercatore 4
5.2 Algoritmi di Clustering e Machine Learning 5

6 Sottosistema di Analisi Statistica e ML **5**
6.1 Clustering Centroidale (MiniBatchKMeans) 5
6.2 Analisi Densimetrica (DBSCAN) 5
6.3 Rappresentazione dei Volumi (Convex Hull) 5

7 UX/UI Design & Interaction Model **5**
7.1 Filosofia Glassmorphism 5
7.2 Sistema di Auto rotazione e Controlli di Navigazione 6
7.3 Modello di Interazione Asincrona e Gestione Multithreading 6

8 Gestione Cache e Performance **7**
8.1 Caching Determinista MD5 7
8.2 Ottimizzazioni di Memoria (LRU) 7

9 Guida all'Uso e Casi d'Uso **7**

10 Conclusioni e Prospettive Evolutive **8**

1 Introduzione ed Analisi del Dominio

Il progetto **Seismic Viewer** nasce dalla necessità di fornire alla comunità scientifica e agli analisti geologici uno strumento performante per la visualizzazione e l'analisi di dataset sismici complessi. In un contesto dove i dati sismici vengono generati con frequenza elevata e precisione millimetrica, la visualizzazione statica non è più sufficiente per comprendere le dinamiche sottostanti alle sequenze di scosse.

L'analisi del dominio ha evidenziato come gli strumenti tradizionali (GIS) siano spesso orientati a una visualizzazione 2D superficiale, trascurando la componente ipocentrale profonda e la dinamica temporale. *Seismic Viewer* si propone di colmare questo divario, integrando le tre dimensioni spaziali con la coordinata temporale (T), creando una vera e propria esperienza di esplorazione 4D.

2 Analisi dei Requisiti

La definizione dei requisiti è stata il punto di partenza per garantire che il software rispondesse sia a necessità tecniche che a standard di usabilità scientifica. Le funzionalità sono state suddivise in base alla loro importanza per il completamento del prototipo funzionale.

2.1 Requisiti Funzionali Primari

Costituiscono il nucleo vitale dell'applicazione:

- **Data Ingestion & Sanitization**: Importazione di dataset in formato CSV con validazione automatica della tipologia dei dati e rimozione del rumore di fondo tramite filtri di magnitudo minima.
- **Motore 3D Ipocentrale**: Visualizzazione spaziale accurata degli eventi sismici con l'asse Z mappato sulla profondità reale.
- **Integrazione Cartografica**: Recupero dinamico di mappe satellitari e topografiche al di sopra degli eventi.
- **Analisi Statistica via Clustering**: Identificazione di sciami sismici tramite algoritmi di Machine Learning (K-Means, DBSCAN) come strumento essenziale per l'interpretazione dei dati.

2.2 Funzionalità Avanzate

Requisiti importanti che arricchiscono l'analisi:

- **Timeline Animata**: Gestione dei flussi temporali con possibilità di osservare l'accumulo degli eventi.

2.3 Estensioni ed Ottimizzazioni

- **Auto rotazione**: Rotazione automatica della camera per un'analisi panoramica senza interventi manuali.

- **Caching Persistente**: Memorizzazione su disco delle mappe per il funzionamento offline parziale.

3 Architettura di Sistema (Tiered Model)

L'architettura di Seismic Viewer è stata progettata seguendo i principi di scalabilità e manutenibilità, utilizzando un modello a tre livelli (*Three-Tier Architecture*).

3.1 Data Layer (Python Core)

Questo livello gestisce l'integrità dei dati. Utilizzando la libreria `pandas`, il software esegue trasformazioni vettoriali ad alta velocità. L'astrazione dal disco è gestita in modo da garantire caricamenti non bloccanti.

3.2 Logic Layer (Bridge API)

Il "cervello" dell'applicazione risiede nel modulo `core/bridge.py`. Questo strato utilizza il pattern **Bridge** per far comunicare l'interprete Python con il frontend JavaScript. È qui che viene gestito lo stato globale dell'applicazione e la sincronizzazione tra il caricamento dei file e la generazione delle viste.

3.3 Presentation Layer (WebView UI)

La visualizzazione è affidata a un motore web accelerato hardware (Chromium), che permette l'uso di *Plotly.js*. Questa scelta garantisce un rendering 3D fluido e un'interfaccia utente interattiva basata su standard web moderni.

4 Infrastruttura di Caricamento (Loader & Pre-computation)

Una delle sfide principali del progetto è la gestione di file CSV che possono contenere decine di migliaia di record. Per evitare rallentamenti durante la navigazione, nel modulo `core/loader.py` è stata implementata una strategia di **Pre-computing**.

4.1 Normalizzazione Temporale

Il sistema non memorizza solo la data, ma calcola preventivamente il valore `dates_num`, che rappresenta il tempo trascorso in giorni dal primo evento osservato. Questo permette al motore di animazione JS di eseguire interpolazioni lineari istantanee senza ricalcolare date complesse a runtime.

4.2 Visual Data Packaging e Ottimizzazione delle Performance

Il caricamento dei dati non consiste in una semplice lettura di un file CSV, ma in un sofisticato processo di **Visual Pre-computation**. Questa tecnica, implementata nel metodo `precompute_visual_data`, è fondamentale per garantire che l'interfaccia JavaScript rimanga fluida (target 60 FPS) anche durante la gestione di dataset massivi.

Il "pacchetto" creato da Python comprende tre elementi critici:

- **Pre-rendering delle Stringhe di Hover**: Per ogni evento sismico, il backend genera preventivamente una stringa HTML complessa contenente data, magnitudo e coordinate. Spostando questa operazione di concatenazione di stringhe da JavaScript a Python, si elimina un enorme carico computazionale dal thread principale del browser, prevenendo micro-lag durante l'interazione con la mappa.
- **Aggregazione Temporale Flessibile**: Per supportare le diverse esigenze analitiche, il sistema pre-calcola indici temporali che abilitano due distinte modalità di visualizzazione:
  - *Cumulativa*: analisi dell'evoluzione storica e volumetrica totale degli eventi fino alla data selezionata.
  - *Finestra Temporale Mobile (Moving Window)*: isolamento di uno specifico intervallo di tempo (es. scivolamento di una finestra di 10 giorni) fondamentale per identificare i trend di migrazione sismica e i pattern di attivazione delle faglie nel tempo.

  Questo approccio strutturato permette al ricercatore di alternare istantaneamente tra le diverse modalità senza ricalcoli onerosi, garantendo un'esplorazione dinamica e fluida del dataset.
- **Metadata Range Synchronization**: Vengono calcolati preventivamente i limiti minimi e massimi di tutte le variabili (magnitudo, profondità, tempo). Questo assicura che le scale cromatiche e i limiti dei filtri dinamici siano sincronizzati istantaneamente al caricamento, evitando flicker o ricalcoli asincroni lato client.

Contemporaneamente, il sistema esegue una normalizzazione spaziale, mappando la profondità sismica sul sistema cartesiano invertendo i segni ($z < 0$), assicurando che l'ipocentro sia correttamente localizzato al di sotto della superficie terrestre.

5 Motore di Visualizzazione e Geospazialità

La componente geografica è gestita dal modulo `core/geo_utils.py`, che si occupa di tradurre il mondo reale in una scena 3D coerente.

5.1 Formalizzazione della Proiezione di Mercatore

Il software implementa una proiezione di Mercatore **inversa** custom, necessaria per mappare correttamente la griglia di tasselli cartografici scaricati sulla geometria 3D. La formula utilizzata per la conversione delle coordinate piane (Y) in coordinate sferiche (Latitudine) è descritta dalla seguente relazione:

$$\phi = \arctan\left(\sinh\left(\pi\left(1 - \frac{2 \cdot y}{n}\right)\right)\right)$$

dove n rappresenta lo zoom level della mappa e y la coordinata normalizzata del tassello. Questo calcolo, eseguito nel modulo `core/geo_utils.py`, assicura la perfetta sovrapposizione tra la mesh del terreno e la posizione reale degli ipocentri sismici.

5.2 Algoritmi di Clustering e Machine Learning

Per l'analisi delle strutture sismogenetiche, il sistema integra algoritmi di *unsupervised learning*:

- **MiniBatchKMeans**: una variante ottimizzata dell'algoritmo K-Means che permette di gestire dataset massivi con un'occupazione di memoria minima, garantendo la reattività del sistema anche su hardware consumer.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**: utilizzato per identificare sciami di forma irregolare basandosi sulla densità volumetrica degli eventi, ignorando automaticamente i terremoti isolati (rumore).

6 Sottosistema di Analisi Statistica e ML

Il sistema non è solo un visualizzatore, ma un analista automatico guidato da algoritmi di Machine Learning situati in `engine.py`.

6.1 Clustering Centroidale (MiniBatchKMeans)

Viene utilizzato l'algoritmo **MiniBatchKMeans** per raggruppare gli eventi in k zone. L'uso della versione "MiniBatch" è una scelta progettuale precisa: essa processa i dati in blocchi casuali, garantendo che anche con 100.000 eventi il calcolo avvenga in meno di un secondo, mantenendo al contempo un'altissima aderenza ai centroidi reali.

6.2 Analisi Densimetrica (DBSCAN)

Per identificare sciami sismici di forma non circolare (tipico delle faglie lineari), viene implementato il **DBSCAN**. Questo algoritmo non richiede il numero di cluster in input, ma lavora sui parametri di densità (Epsilon e MinSamples), permettendo di isolare i terremoti "rumore" (isolati) dalla sequenza principale.

6.3 Rappresentazione dei Volumi (Convex Hull)

Per rendere visibile l'influenza spaziale di un cluster, il software calcola il **Convex Hull** (Inviluppo Convesso). Utilizzando l'algoritmo Quickhull, viene generata una mesh 3D che "avvolge" tutti i punti del cluster, dando la percezione del volume di roccia interessato dalla sequenza sismica.

7 UX/UI Design & Interaction Model

L'interfaccia utente è progettata per essere "viva" e reattiva, utilizzando i canoni estetici del **Glassmorphism**.

7.1 Filosofia Glassmorphism

I menu e le dashboard utilizzano sfondi semi-trasparenti con effetto di sfocatura dello sfondo (*backdrop-filter blur*). Questa scelta garantisce che i controlli non occupino uno

spazio "cieco", permettendo di vedere i dati sismici che scorrono dietro ai pannelli di controllo.

7.2 Sistema di Auto rotazione e Controlli di Navigazione

Per massimizzare l'usabilità e la precisione dell'analisi spaziale, il sistema integra funzionalità di navigazione avanzata:

- **Auto rotazione**: permette di osservare la nube di punti sismici da ogni angolazione, facilitando l'identificazione di pendenze o allineamenti di faglie non visibili da una prospettiva fissa.
- **Controlli UI Diretti**: pulsanti posizionati nella parte superiore e laterale del grafico per una gestione immediata senza l'uso del mouse.
- **Prospettive Predefinite**: possibilità di "saltare" istantaneamente tra viste dall'alto, laterali e prospettiche per un cambio di contesto rapido.
- **Toggle Dinamico**: attivazione e disattivazione della rotazione automatica in tempo reale per soffermarsi su specifici dettagli del volume sismico.

7.3 Modello di Interazione Asincrona e Gestione Multithreading

L'efficienza della *User Experience* (UX) all'interno di Seismic Viewer è garantita dall'adozione di un paradigma di interazione asincrona, volto a mitigare le latenze di risposta del thread principale durante l'esecuzione di task ad alto onere computazionale (p.es. la generazione di mesh ad alta risoluzione o l'analisi statistica tramite clustering di dataset massivi).

L'architettura del sistema si fonda su una gestione strutturata della concorrenza articolata in tre livelli:

- **Delegazione Computazionale (Worker Threads)**: Ogni richiesta generata dal frontend viene intercettata dal modulo `core/bridge.py` e instradata su un thread indipendente mediante la libreria `threading`. L'implementazione di thread di tipo "daemon" assicura la sincronizzazione del ciclo di vita dei processi in background con quello dell'applicazione principale, garantendo una corretta deallocazione delle risorse al termine della sessione.
- **Propagazione dello Stato via Callback**: Durante l'esecuzione dei processi asincroni, il backend Python comunica costantemente lo stato di avanzamento al motore di rendering. Tale interscambio avviene tramite l'invocazione del metodo `evaluate_js`, che inietta istruzioni JavaScript all'interno del motore Chromium per l'aggiornamento dinamico del *Progress Hub*. Questo meccanismo di feedback continuo minimizza l'incertezza informativa per l'utente, mantenendo alta la percezione di reattività del sistema.
- **Sincronizzazione Event-Driven**: Per preservare l'integrità del rendering, le transizioni di stato tra i diversi moduli dell'applicazione sono regolate da meccanismi di temporizzazione (`threading.Timer`). Questo approccio consente di regolare la latenza necessaria affinché il server HTTP locale completi le operazioni di I/O sui file temporanei, assicurando transizioni atomiche e prive di anomalie visive.

Tale suddivisione dei compiti permette di mantenere il *Main Event Loop* costantemente disponibile per l'elaborazione degli input utente, quali la navigazione spaziale o la manipolazione dei filtri dinamici, delegando la complessità computazionale alla logica asincrona sottostante.

8 Gestione Cache e Performance

Per garantire l'usabilità professionale, Seismic Viewer implementa tecniche sofisticate di gestione delle risorse nel package `core/`.

8.1 Caching Determinista MD5

Le mesh del terreno vengono calcolate una sola volta per ogni area geografica. Il sistema calcola un hash **MD5** basato sulle coordinate geografiche; se un hash corrispondente è trovato in `resources/.map_cache`, la mesh viene caricata istantaneamente tramite **Pickle**, risparmiando banda e tempo di CPU.

8.2 Ottimizzazioni di Memoria (LRU)

Le funzioni di generazione mesh utilizzano un decoratore `lru_cache` (Least Recently Used) a livello di sistema. Questo assicura che se l'utente torna a una visualizzazione precedente, essa venga caricata istantaneamente dalla RAM anziché dal disco o dal web.

9 Guida all'Uso e Casi d'Uso

L'applicazione Seismic Viewer è stata progettata per supportare il ricercatore in diverse fasi dell'analisi sismologica. Di seguito vengono descritte le principali modalità operative e di visualizzazione integrate:

- **Analisi Planimetrica 2D**: Fornisce una vista planimetrica classica della distribuzione degli epicentri. È lo strumento primario per una rapida valutazione dell'estensione geografica della sequenza e per l'allineamento con la cartografia stradale o urbana.
- **Modellazione Volumetrica 3D**: Integra i dati ipocentrali con la morfologia digitale del terreno. Questa vista permette di apprezzare la profondità degli eventi e l'inclinazione delle strutture sismogenetiche, visualizzando i terremoti al di sotto della mesh superficiale.
- **Simulazione Temporale 4D**: Attraverso la modalità di animazione, il sistema permette di osservare la propagazione spazio-temporale dell'energia. Questa analisi è fondamentale per distinguere la migrazione dei fluidi o le attivazioni a catena di diverse faglie nel tempo.
- **Analisi Avanzata tramite Clustering**: Implementa algoritmi di Machine Learning per l'identificazione automatica di cluster sismici. Tale approccio facilita la discriminazione analitica di differenti sotto-sequenze e l'identificazione di strutture sismogenetiche indipendenti, basandosi su rigorosi criteri di densità spaziale e temporale dei dati ipocentrali.

Il workflow tipico prevede il caricamento di un dataset (es. Terremoti dei Campi Flegrei), l'analisi preliminare in 2D e il successivo approfondimento volumetrico e statistico per caratterizzare la geometria della sorgente sismica.

10 Conclusioni e Prospettive Evolutive

In conclusione, Seismic Viewer si configura come una struttura modulare e altamente scalabile dedicata alla ricerca geofisica avanzata. Il disaccoppiamento logico tra i processi di elaborazione dati del backend e le componenti di visualizzazione del frontend abilita future integrazioni con flussi di dati in tempo reale via protocollo WebSocket e l'incorporazione di complessi algoritmi predittivi basati su reti neurali profonde (Deep Learning). Il software riesce a coniugare un elevato rigore scientifico e computazionale con un'interfaccia utente ottimizzata secondo i principi della moderna interazione uomo-macchina, ponendosi come un efficace strumento di supporto alla ricerca sismologica e all'analisi del rischio geologico.